

Codigo JSX comentado - Gestion de usuarios Finflow

Evidencia GA7-220501096-AA3-EV01. El lado izquierdo muestra App.jsx y el derecho explica la finalidad de cada linea.

Componente React: App.jsx

La aplicacion emplea useState para manejar el formulario, la lista de usuarios, la edicion y los mensajes. Los comentarios presentes en el archivo fuente se transcriben a continuacion.

Linea y codigo JSX	Explicacion
001 import { useState } from 'react'; // Importa el Hook que permite guardar datos que cambian en pantalla.	Importa el Hook que permite guardar datos que cambian en pantalla.
002	Linea en blanco que separa bloques logicos del componente.
003 const usuarioInicial = { documento: '', nombres: '', apellidos: '', correo: '', telefono: '' }; // Define un formulario vacio reutilizable.	Define un formulario vacío reutilizable.
004	Linea en blanco que separa bloques logicos del componente.
005 const datosIniciales = [// Declara datos de ejemplo para mostrar la consulta al abrir el módulo.	Declara datos de ejemplo para mostrar la consulta al abrir el módulo.
006 { id: 1, documento: '1000000001', nombres: 'Ana', apellidos: 'Gómez', correo: 'ana@finflow.com', telefono: '3001234567' } // Crea el primer usuario de demostración.	Crea el primer usuario de demostración.
007];	Instruccion que forma parte de la estructura del componente React.
008	Linea en blanco que separa bloques logicos del componente.
009 function App() { // Declara el componente principal que React mostrará en la interfaz.	Declara el componente principal que React mostrará en la interfaz.
010 const [usuarios, setUsuarios] = useState(datosIniciales); // Guarda la lista de usuarios registrados.	Guarda la lista de usuarios registrados.
011 const [formulario, setFormulario] = useState(usuarioInicial); // Guarda lo escrito actualmente en los campos.	Guarda lo escrito actualmente en los campos.
012 const [idEdicion, setIdEdicion] = useState(null); // Guarda el identificador del usuario que se está editando.	Guarda el identificador del usuario que se está editando.
013 const [mensaje, setMensaje] = useState(''); // Guarda los mensajes que se muestran a la persona usuaria.	Guarda los mensajes que se muestran a la persona usuaria.
014	Linea en blanco que separa bloques logicos del componente.
015 const actualizarCampo = (evento) => { // Define la función que responde cuando cambia un campo del formulario.	Define la función que responde cuando cambia un campo del formulario.
016 const { name, value } = evento.target; // Obtiene el nombre del campo y el texto ingresado.	Obtiene el nombre del campo y el texto ingresado.
017 setFormulario({ ...formulario, [name]: value }); // Copia el formulario anterior y actualiza solo el campo modificado.	Copia el formulario anterior y actualiza solo el campo modificado.
018 }; // Finaliza la función de actualización del formulario.	Finaliza la función de actualización del formulario.
019	Linea en blanco que separa bloques logicos del componente.
020 const limpiarFormulario = () => { // Declara una función para volver el formulario a su estado inicial.	Declara una función para volver el formulario a su estado inicial.
021 setFormulario(usuarioInicial); // Limpia todos los valores escritos en los campos.	Limpia todos los valores escritos en los campos.

Línea y código JSX	Explicación
022 setIdEdicion(null); // Indica que ya no hay un usuario seleccionado para edición.	Indica que ya no hay un usuario seleccionado para edición.
023 }; // Finaliza la función que limpia el formulario.	Finaliza la función que limpia el formulario.
024	Línea en blanco que separa bloques lógicos del componente.
025 const guardarUsuario = (evento) => { // Declara la función que procesa el envío del formulario.	Declara la función que procesa el envío del formulario.
026 evento.preventDefault(); // Evita que el navegador recargue la página al enviar el formulario.	Evita que el navegador recargue la página al enviar el formulario.
027 const datosLimpios = Object.fromEntries(Object.entries(formulario).map(([clave, valor]) => [clave, valor.trim()])); // Elimina espacios innecesarios en cada dato.	Elimina espacios innecesarios en cada dato.
028 const repetido = usuarios.some((usuario) => usuario.id !== idEdicion && (usuario.documento === datosLimpios.documento usuario.correo.toLowerCase() === datosLimpios.correo.toLowerCase())); // Busca documento o correo repetidos.	Busca documento o correo repetidos.
029	Línea en blanco que separa bloques lógicos del componente.
030 if (repetido) { // Comprueba si se encontró un dato que ya pertenece a otro usuario.	Comprueba si se encontró un dato que ya pertenece a otro usuario.
031 setMensaje('No se puede guardar: el documento o el correo ya están registrados.');// Informa claramente la regla de validación.	Informa claramente la regla de validación.
032 return; // Detiene la operación para no duplicar información.	Detiene la operación para no duplicar información.
033 } // Finaliza la condición de datos repetidos.	Finaliza la condición de datos repetidos.
034	Línea en blanco que separa bloques lógicos del componente.
035 if (idEdicion) { // Comprueba si el formulario se está usando para editar un usuario existente.	Comprueba si el formulario se está usando para editar un usuario existente.
036 setUsuarios(usuarios.map((usuario) => usuario.id === idEdicion ? { ...datosLimpios, id: idEdicion } : usuario)); // Reemplaza solo el usuario seleccionado.	Reemplaza solo el usuario seleccionado.
037 setMensaje('Usuario actualizado correctamente.');// Muestra el resultado de la actualización.	Muestra el resultado de la actualización.
038 } else { // Ejecuta esta parte cuando se desea crear un nuevo usuario.	Ejecuta esta parte cuando se desea crear un nuevo usuario.
039 setUsuarios([...usuarios, { ...datosLimpios, id: Date.now() }]); // Agrega el nuevo usuario con un identificador único.	Agrega el nuevo usuario con un identificador único.
040 setMensaje('Usuario registrado correctamente.');// Muestra el resultado de la inserción.	Muestra el resultado de la inserción.
041 } // Finaliza la decisión entre crear y actualizar.	Finaliza la decisión entre crear y actualizar.
042	Línea en blanco que separa bloques lógicos del componente.
043 limpiarFormulario(); // Restablece el formulario después de guardar correctamente.	Restablece el formulario después de guardar correctamente.
044 }; // Finaliza la función de guardado.	Finaliza la función de guardado.
045	Línea en blanco que separa bloques lógicos del componente.
046 const editarUsuario = (usuario) => { // Declara la función que prepara un registro para ser modificado.	Declara la función que prepara un registro para ser modificado.
047 setFormulario({ documento: usuario.documento, nombres: usuario.nombres, apellidos: usuario.apellidos, correo: usuario.correo, telefono: usuario.telefono }); // Copia los datos del usuario en el formulario.	Copia los datos del usuario en el formulario.
048 setIdEdicion(usuario.id); // Guarda el identificador para saber qué registro actualizar.	Guarda el identificador para saber qué registro actualizar.
049 setMensaje(`Editando a \${usuario.nombres} \${usuario.apellidos}.`); // Indica qué usuario está en modo edición.	Indica qué usuario está en modo edición.

Línea y código JSX	Explicación
050 }; // Finaliza la función de edición.	Finaliza la función de edición.
051	Línea en blanco que separa bloques lógicos del componente.
052 const eliminarUsuario = (id) => { // Declara la función que elimina un usuario de la lista.	Declara la función que elimina un usuario de la lista.
053 const confirmar = window.confirm('¿Desea eliminar este usuario?'); // Solicita confirmación para evitar eliminaciones accidentales.	Solicita confirmación para evitar eliminaciones accidentales.
054 if (!confirmar) return; // Detiene la operación si la persona cancela la confirmación.	Detiene la operación si la persona cancela la confirmación.
055 setUsuarios(usuarios.filter((usuario) => usuario.id !== id)); // Conserva todos los usuarios excepto el seleccionado.	Conserva todos los usuarios excepto el seleccionado.
056 setMensaje('Usuario eliminado correctamente.');	Informa que la eliminación se completó.
057 if (id === idEdicion) limpiarFormulario(); // Limpia el formulario si se eliminó el usuario que se estaba editando.	Limpia el formulario si se eliminó el usuario que se estaba editando.
058 }; // Finaliza la función de eliminación.	Finaliza la función de eliminación.
059	Línea en blanco que separa bloques lógicos del componente.
060 return (// Devuelve el contenido JSX que se visualizará en el navegador.	Devuelve el contenido JSX que se visualizará en el navegador.
061 <main className="contenedor"> { /* Agrupa todo el contenido principal del módulo. */ }	Agrupar todo el contenido principal del módulo.
062 <header> { /* Agrupa el título y la explicación del módulo. */ }	Agrupar el título y la explicación del módulo.
063 <p className="etiqueta">FINFLOW · REACT</p> { /* Identifica la tecnología usada en la evidencia. */ }	Identifica la tecnología usada en la evidencia.
064 Gestión de usuarios { /* Muestra el título principal de la interfaz. */ }	Muestra el título principal de la interfaz.
065 Registre y administre los datos de los usuarios del sistema financiero. { /* Explica la finalidad funcional de la pantalla. */ }	Explica la finalidad funcional de la pantalla.
066 </header> { /* Cierra el encabezado. */ }	Cierra el encabezado.
067	Línea en blanco que separa bloques lógicos del componente.
068 {mensaje && <p className="mensaje" role="status">{mensaje}</p> } { /* Presenta mensajes solo cuando existe alguno. */ }	Presenta mensajes solo cuando existe alguno.
069	Línea en blanco que separa bloques lógicos del componente.
070 <section className="tarjeta"> { /* Agrupa el formulario dentro de una tarjeta visual. */ }	Agrupar el formulario dentro de una tarjeta visual.
071 {idEdicion ? 'Actualizar usuario' : 'Registrar usuario'} { /* Cambia el título según la operación. */ }	Cambia el título según la operación.
072 <form onSubmit={guardarUsuario}> { /* Ejecuta guardarUsuario al enviar datos válidos. */ }	Ejecuta guardarUsuario al enviar datos válidos.
073 <Campo etiqueta="Documento de identidad" nombre="documento" valor={formulario.documento} alCambiar={actualizarCampo} /> { /* Renderiza el campo documento. */ }	Renderiza el campo documento.
074 <Campo etiqueta="Nombres" nombre="nombres" valor={formulario.nombres} alCambiar={actualizarCampo} /> { /* Renderiza el campo nombres. */ }	Renderiza el campo nombres.
075 <Campo etiqueta="Apellidos" nombre="apellidos" valor={formulario.apellidos} alCambiar={actualizarCampo} /> { /* Renderiza el campo apellidos. */ }	Renderiza el campo apellidos.
076 <Campo etiqueta="Correo electrónico" nombre="correo" tipo="email" valor={formulario.correo} alCambiar={actualizarCampo} /> { /* Renderiza el campo correo con validación HTML. */ }	Renderiza el campo correo con validación HTML.

Línea y código JSX	Explicación
077 <Campo etiqueta="Teléfono de contacto" nombre="telefono" valor={formulario.telefono} alCambiar={actualizarCampo} /> { /* Renderiza el campo teléfono. */ }	Renderiza el campo teléfono.
078 <div className="acciones"> { /* Agrupa los botones del formulario. */ }	Agrupar los botones del formulario.
079 <button type="submit">{idEdicion ? 'Guardar cambios' : 'Registrar usuario'}</button> { /* Envía el formulario. */ }	Envía el formulario.
080 {idEdicion && <button type="button" className="secundario" onClick={limpiarFormulario}>Cancelar</button> } { /* Permite abandonar la edición. */ }	Permite abandonar la edición.
081 </div> { /* Cierra el grupo de botones. */ }	Cierra el grupo de botones.
082 </form> { /* Cierra el formulario. */ }	Cierra el formulario.
083 </section> { /* Cierra la tarjeta del formulario. */ }	Cierra la tarjeta del formulario.
084	Línea en blanco que separa bloques lógicos del componente.
085 <section className="tarjeta"> { /* Agrupa la tabla de consulta de usuarios. */ }	Agrupar la tabla de consulta de usuarios.
086 Usuarios registrados ({usuarios.length}) { /* Muestra la cantidad actual de registros. */ }	Muestra la cantidad actual de registros.
087 <div className="tabla-responsive"> { /* Permite desplazar la tabla en pantallas pequeñas. */ }	Permite desplazar la tabla en pantallas pequeñas.
088 <table> { /* Inicia la tabla con los datos consultados. */ }	Inicia la tabla con los datos consultados.
089 <thead><tr><th>Documento</th><th>Nombres</th><th>Apellidos</th><th>Correo</th><th>Teléfono</th><th>Acciones</th></tr></thead> { /* Define los encabezados de columna. */ }	Define los encabezados de columna.
090 <tbody> { /* Inicia el cuerpo de la tabla. */ }	Inicia el cuerpo de la tabla.
091 {usuarios.map((usuario) => (// Recorre cada usuario y genera una fila.	Recorre cada usuario y genera una fila.
092 <tr key={usuario.id}> { /* Asigna una clave única a la fila para React. */ }	Asigna una clave única a la fila para React.
093 <td>{usuario.documento}</td><td>{usuario.nombres}</td><td>{usuario.apellidos}</td><td>{usuario.correo}</td><td>{usuario.telefono}</td> { /* Muestra los datos del usuario. */ }	Muestra los datos del usuario.
094 <td><button className="editar" onClick={() => editarUsuario(usuario)}>Editar</button><button className="eliminar" onClick={() => eliminarUsuario(usuario.id)}>Eliminar</button></td> { /* Incluye las acciones de actualización y eliminación. */ }	Incluye las acciones de actualización y eliminación.
095 { /* Cierra la fila del usuario. */ }</tr>	Cierra la fila del usuario.
096 }}} { /* Finaliza el recorrido de usuarios. */ }	Finaliza el recorrido de usuarios.
097 </tbody> { /* Cierra el cuerpo de la tabla. */ }	Cierra el cuerpo de la tabla.
098 </table> { /* Cierra la tabla. */ }	Cierra la tabla.
099 </div> { /* Cierra el contenedor adaptable de la tabla. */ }	Cierra el contenedor adaptable de la tabla.
100 </section> { /* Cierra la tarjeta de consulta. */ }	Cierra la tarjeta de consulta.
101 { /* Cierra el contenido principal. */ }</main>	Cierra el contenido principal.
102); // Finaliza el JSX devuelto.	Finaliza el JSX devuelto.
103 } // Finaliza el componente App.	Finaliza el componente App.
104	Línea en blanco que separa bloques lógicos del componente.

Línea y código JSX	Explicación
105 function Campo({ etiqueta, nombre, valor, alCambiar, tipo = 'text' }) { // Declara un componente reutilizable para cada campo del formulario.	Declara un componente reutilizable para cada campo del formulario.
106 return (// Devuelve la etiqueta y el control de entrada.	Devuelve la etiqueta y el control de entrada.
107 <label> { /* Relaciona el texto descriptivo con el campo de entrada. */ }	Relaciona el texto descriptivo con el campo de entrada.
108 {etiqueta} { /* Muestra el nombre legible del campo. */ }	Muestra el nombre legible del campo.
109 <input type={tipo} name={nombre} value={valor} onChange={alCambiar} required /> { /* Captura el dato y comunica cambios a App. */ }	Captura el dato y comunica cambios a App.
110 { /* Cierra la etiqueta del campo. */ }</label>	Cierra la etiqueta del campo.
111); // Finaliza el JSX del componente Campo.	Finaliza el JSX del componente Campo.
112 } // Finaliza el componente reutilizable.	Finaliza el componente reutilizable.
113	Línea en blanco que separa bloques lógicos del componente.
114 export default App; // Exporta App para que main.jsx pueda renderizarlo.	Exporta App para que main.jsx pueda renderizarlo.

Conclusiones tecnicas

El componente cumple las operaciones de registrar, consultar, actualizar y eliminar en la interfaz. Se usa un componente Campo para evitar repetir codigo y una validacion para impedir documentos o correos duplicados. La informacion se conserva en el estado de React mientras la aplicacion esta abierta.